# TP 2 : Simulations et clustering spatial sur GPU

Boids (grille, forces, intégration), puis particules de deux types

Programmation parallèle GPU : OpenGL 4.3 / GLSL 4.30

---

## Objectifs

— simuler des **boids** (séparation, alignement, cohésion) en **trois passes** : grille et listes, forces, intégration.

— construire cette **structure spatiale entièrement sur GPU** : grille de cellules et listes chaînées, avec des opérations atomiques.

— réutiliser les mêmes trois passes pour des **particules de deux types** (attraction / répulsion), et les comparer à une référence exhaustive.

## Prérequis

TP1 (SSBO, indices, dispatch, barrières, ping-pong). Notion de complexité. Aucune connaissance en simulation physique n'est supposée au-delà d'une intégration d'Euler.

## 1 Environnement et éléments fournis

Trois squelettes, compilés en C++11 avec gcc et CMake comme au TP1 : `tp2/exercice1` (boids), `tp2/exercice2` (particules, référence exhaustive) puis `tp2/exercice3` (même modèle, version spatiale). Ils fournissent le contexte OpenGL 4.3, les utilitaires de buffers, ainsi que :

— drawPoints dans les trois exercices.

— initBoids dans l'exercice 1, initParticles dans les exercices 2 et 3.

— clearSSBO et les buffers de listes dans les exercices 1 et 3.

**Interface du squelette**

```
// Remise a zero d'un SSBO (remplissage d'une valeur 32 bits).
void clearSSBO(GLuint buffer, uint32_t value);

void drawPoints(GLuint particleBuffer, GLsizei count);
void initParticles(std::vector<Particle>& dst, int n, float L);
void initBoids(std::vector<Particle>& dst, int n, float L);
```

La mesure du temps se fait avec un simple chronomètre C++ (std::chrono), avec un glFinish() avant la lecture de l'horloge (cf. TP1).

# 2 Conventions communes aux deux simulations

## 2.1 Domaine borné et états

Les deux simulations vivent dans une boîte alignée sur les axes, d'origine **o** et de côté $L$ (par exemple $L = 64$). Les particules et les boids qui sortent de la boîte sont **réfléchis** aux parois : la composante de vitesse normale est inversée et la position ramenée à l'intérieur. Ce choix vient de la grille du TP : un tableau dense, de taille fixe, indexé par des coordonnées de cellules. Une particule hors de la boîte n'a pas de case dans ce tableau. Un domaine non borné reste accessible avec une autre structure, par exemple un hachage des coordonnées de cellules ou une grille creuse. Ce n'est pas l'objet du TP.

Chaque simulation maintient deux états (ping-pong) :

$$\texttt{current}\ \text{(lecture seule)} \longrightarrow \text{compute} \longrightarrow \texttt{next}\ \text{(écriture seule)},$$

échangés à la fin de chaque pas de temps.

## 2.2 Représentation GPU

```
struct Particle {
    vec4 posType; // xyz : position, w : type (+1.0 ou -1.0)
    vec4 vel;     // xyz : vitesse, w : inutilise (0.0)
};
```

Côté C++, la structure correspondante utilise alignas(16) et quatre float par champ : l'agencement std430 des tableaux de vec4 est alors direct, sans rembourrage.

**Attention**

N'utilisez jamais de tableau de vec3 en std430 sans vérifier le rembourrage : chaque élément occupe 16 octets. Le TP impose des vec4 partout.

## 2.3 Rayon d'interaction fini

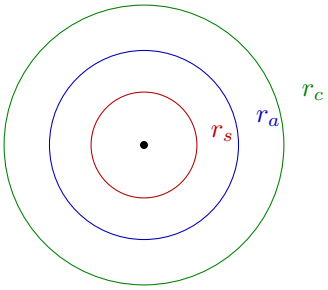
Les simulations de ce TP utilisent une loi à **rayon d'interaction fini** $R$ : deux particules n'interagissent que si leur distance est inférieure à $R$. Ce choix est essentiel : c'est lui qui rend la structure spatiale *correcte* (elle ne cherche que des voisins proches) et comparable à une référence exhaustive *qui applique la même loi tronquée*.

**Attention**

La version exhaustive de référence et la version par grille doivent implémenter **exactement la même loi tronquée**, mêmes conditions aux limites comprises. Une comparaison entre deux lois différentes ne valide rien.

# 3 Exercice 1 : Boids en trois passes

Squelette : `tp2/exercice1`. Trois shaders à écrire, dans cet ordre : `boids_build.comp` (listes), `boids_forces.comp` (forces), `boids_integrate.comp` (intégration). Aucun ne fait le travail des deux autres.




séparation, alignement, cohésion

Chaque boid possède une position et une vitesse. Trois règles produisent une accélération, avec trois rayons distincts :

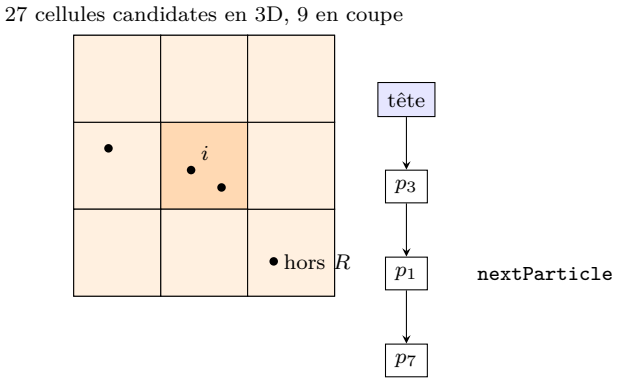
— **séparation** (rayon $r_s$, petit) : s'écarter des voisins trop proches.

— **alignement** (rayon $r_a$) : aligner sa vitesse sur la vitesse moyenne des voisins.

— **cohésion** (rayon $r_c$) : se diriger vers le centre de masse des voisins.

Chaque règle fournit une contribution pondérée (coefficients $\alpha_s, \alpha_a, \alpha_c$), sommées dans l'accélération. La vitesse est bornée par $v_{max}$. L'intégration est un Euler semi-implicite, avec réflexion aux parois : elle est écrite seule, dans la troisième passe.

## 3.1 Étape 1 : grille et listes




La grille régulière remplace un parcours de tous les autres boids. Le côté de cellule est $h = R$, avec $R = \max(r_s, r_a, r_c)$ : le plus grand rayon tient dans le voisinage de 27 cellules. Chaque boid ne compare ensuite qu'aux boids de sa cellule et des 26 cellules voisines.

### 3.1.1 Calcul de la cellule

Pour un boid en $\mathbf{x}_i$, dans une grille de dimensions $(n_x, n_y, n_z)$ :

$$\mathbf{c}_i = \left\lfloor \frac{\mathbf{x}_i - \mathbf{o}}{h} \right\rfloor, \qquad cellID = c_x + n_x\,(c_y + n_y\,c_z)\,. \tag{1}$$

Justifier pourquoi $h = R$ garantit que tous les voisins à distance inférieure à $R$ se trouvent dans les 27 cellules centrées sur celle de $i$.

1. Quelle valeur minimale de $h$ garantit que les voisins des *trois* règles sont dans ces 27 cellules ?
2. Que se passe-t-il si $h < \max(r_s, r_a, r_c)$ ? Donner le nombre de cellules à explorer dans chaque direction en fonction de $h$ et du rayon.

### 3.1.2 Listes chaînées d'indices

La grille ne stocke pas les boids. Elle stocke, pour chaque cellule, **la liste des indices** de ceux qui s'y trouvent. Deux tableaux suffisent, parce qu'un boid n'appartient qu'à une seule cellule :

```
layout(std430, binding = 4) buffer Heads {
    int cellHead[];   // une entree par cellule
};
layout(std430, binding = 5) buffer Next {
    int nextParticle[]; // une entree par boid
};
```

— cellHead[c] est l'indice du premier boid de la cellule $c$, ou $-1$ si la cellule est vide.

— nextParticle[i] est l'indice du boid qui suit $i$ dans la même cellule, ou $-1$ en fin de liste.

Les valeurs sont des indices dans le tableau de boids, pas des pointeurs. La sentinelle $-1$ termine chaque chaîne. Par exemple, si la cellule orange de la figure contient les boids 3, 1 puis 7 :

| indice | 0 | 1 | 3 | 7 |
|---|---|---|---|---|
| nextParticle | · | 7 | 1 | $-1$ |

cellHead[cellule] $= 3$

On lit la cellule en partant de la tête et en suivant nextParticle jusqu'à $-1$ : $3 \to 1 \to 7$. L'ordre dans la liste est celui des insertions, pas un ordre spatial.

```
for (int j = cellHead[cellID]; j != -1; j = nextParticle[j]) {
    // le boid j est dans cette cellule
}
```

Cette représentation n'impose aucune capacité par cellule : une cellule peut contenir 0, 1 ou $N$ boids, et le tableau nextParticle a exactement $N$ cases. Un tableau *cell* $\times$ *capacité* obligerait à choisir cette capacité à l'avance, et à la gaspiller dans les cellules vides ou à la dépasser dans un amas.

L'insertion se fait **en tête**, sans déplacer les indices déjà en liste. Deux cases seulement changent : la tête de la cellule, et le lien du boid qui arrive. Si cellHead valait 1 et que le boid 3 s'insère, atomicExchange écrit 3 à la place de 1 et renvoie l'ancienne valeur 1, rangée dans nextParticle[3]. La chaîne $1 \to 7 \to -1$ n'est pas réécrite : 3 pointe vers 1, qui pointe toujours vers 7.

Plusieurs boids de la même cellule peuvent s'insérer en même temps. L'échange atomique sérialise les écritures sur la tête : chacun récupère la tête précédente et la range dans son propre nextParticle. Sans lui, deux invocations liraient la même tête et l'une écraserait l'autre, qui disparaîtrait de la liste.

### 3.1.3 Reconstruction complète

Insérer est bon marché. **Retirer** ne l'est pas. Pour sortir le boid $i$ de sa cellule, il faut réécrire le lien qui pointe vers lui : la tête, s'il est en premier, sinon le nextParticle de son prédécesseur. Une liste simplement chaînée ne connaît pas ce prédécesseur. Le trouver demande de reparcourir la cellule, et deux invocations qui retirent en même temps ne peuvent pas le faire par un seul atomicExchange.

Le TP reconstruit donc tout : remise à zéro des têtes, puis une insertion par boid. C'est $O(N)$ écritures atomiques, indépendantes de l'endroit où les boids sont allés.

### 3.1.4 Shader de construction

Une invocation par boid :

1. lire la position dans l'état *courant*.

2. calculer cellID.
3. s'insérer en tête de liste :

```
int previous = atomicExchange(
    cellHead[cellID], int(particleID));
nextParticle[particleID] = previous;
```

**Attention**

1. Les têtes doivent être réinitialisées à $-1$ **avant** chaque reconstruction : dispatch séparé (ou clearSSBO fourni), puis barrière SSBO avant la construction.
2. Le shader de construction ne doit **jamais lire** les listes : elles ne sont cohérentes qu'après la fin du dispatch entier.
3. Les listes ne sont valables que pour *les positions utilisées à la construction*. Les passes suivantes lisent ces positions ; elles n'écrivent le nouvel état qu'à l'intégration.

Avec des parois réfléchissantes, les boids ne sortent jamais du domaine : les cellules candidates hors de la grille sont simplement **exclues** (pas de repli modulo). Une position exactement sur la borne supérieure du domaine est ramenée à la dernière cellule. Ces règles doivent être identiques dans la construction et dans le calcul des forces.

Pour vérifier les listes avant d'écrire une force, implémenter :

```
// Nombre de boids a distance < radius de `p`,
// en excluant `excludeID` (-1 pour n'exclure personne).
uint countWithinRadius(vec3 p, float radius, int excludeID);
```

Le paramètre excludeID est indispensable : sans exclusion, le plus proche voisin d'un boid est lui-même.

### 3.2 Étape 2 : forces

Une invocation par boid. Ce shader **lit** l'état courant et les listes, et **écrit** un tampon de forces. Il ne modifie ni les positions, ni les vitesses, ni les listes.

```
layout(std430, binding = 6) writeonly buffer Forces {
    vec4 force[]; // xyz : acceleration, w : 0
};
```

1. déterminer les 27 cellules candidates, en excluant celles hors de la grille.
2. parcourir les listes correspondantes.
3. pour chaque voisin $j \neq i$, tester la distance exacte contre $r_s$, $r_a$ et $r_c$ : $\|\mathbf{x}_j - \mathbf{x}_i\|^2 < r^2$.
4. accumuler le vecteur de séparation, la vitesse moyenne et le centre de masse, chacun sur son rayon.
5. combiner les trois contributions pondérées et écrire `force[i]`.

Tant que cette passe n'est pas finie, le tampon de forces n'est pas cohérent : l'intégration est un dispatch séparé, après une barrière SSBO.

### 3.3 Étape 3 : intégration

Une invocation par boid. Ce shader lit l'état courant et `force[i]`, écrit l'état suivant, et ne relit pas les listes.

$$\mathbf{v}_i^{t+\Delta t} = \mathbf{v}_i^t + \Delta t\, \mathbf{a}_i, \tag{2}$$

$$\mathbf{x}_i^{t+\Delta t} = \mathbf{x}_i^t + \Delta t\, \mathbf{v}_i^{t+\Delta t}. \tag{3}$$

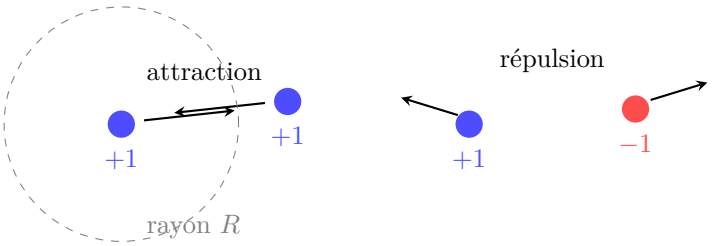
Puis borner $\|\mathbf{v}\|$ par $v_{max}$, réfléchir aux parois, et écrire `next`. Afficher les boids orientés (petits triangles dans le plan $xy$) depuis le SSBO, sans rapatrier l'état sur CPU.

### 3.4 Enchaînement

| | |
|---|---|
| dispatch 0 | réinitialisation des têtes ($-1$) |
| barrière | SSBO |
| dispatch 1 | construction des listes |
| barrière | SSBO |
| dispatch 2 | forces (écriture de `force[]`) |
| barrière | SSBO |
| dispatch 3 | intégration et écriture de `next` |
| hôte | échange `current` / `next` |

# 4 Exercice 2 : Particules de deux types

Squelette : `tp2/exercice2`. Le shader `particles.comp` recopie l'état : l'interaction et l'intégration sont à écrire. C'est la référence exhaustive, en $O(N^2)$. La version par grille est l'exercice 3.




## 4.1 Le modèle Janus

Le modèle cosmologique Janus (J.-P. Petit) fait coexister deux populations de masses de signes opposés. Dans la limite newtonienne que l'on retient ici, le signe de l'interaction ne dépend que du produit des types :

— deux particules de **même signe** s'attirent.

— deux particules de **signes opposés** se repoussent.

Chaque population s'agrège, et les deux populations se séparent. En principe, on doit observer la formation d'un **grand attracteur** de particules positives et d'un **grand répulseur** de particules négatives, analogues aux structures vues dans les relevés cosmiques (grand attracteur, grand répulseur). Dans d'autres modèles, le rôle tenu ici par les particules négatives est celui de la matière noire.

On se simplifie la vie pour le calcul : types $q_i \in \{+1, -1\}$, rayon d'interaction fini, régularisation à courte distance, boîte réfléchissante et intégrateur d'Euler. La loi de signes reste celle du modèle.

On simule $N$ particules de type $q_i \in \{+1, -1\}$ (masse 1), avec $K > 0$ et $\epsilon^2 > 0$ :

$$\mathbf{a}_i \;=\; K \sum_{\substack{j \neq i \\ \|\mathbf{x}_j - \mathbf{x}_i\| < R}} q_i\, q_j \, \frac{\mathbf{x}_j - \mathbf{x}_i}{\left(\|\mathbf{x}_j - \mathbf{x}_i\|^2 + \epsilon^2\right)^{3/2}}. \tag{4}$$

Avec cette convention :

— même type ($q_i q_j = +1$) : l'accélération pointe *vers* $j$ (attraction).

— types différents ($q_i q_j = -1$) : elle pointe *hors* de $j$ (répulsion).

Il n'y a **ni collision ni contrainte de non-pénétration** : deux particules peuvent se traverser. Le terme $\epsilon^2$ borne le dénominateur quand deux particules se confondent : sans lui, la distance nulle provoque une division par zéro, et une distance très petite fait exploser l'accélération au-delà de ce que la précision flottante peut intégrer.

## 4.2 Intégration temporelle

Utiliser l'**Euler semi-implicite** (symplectique) :

$$\mathbf{v}_i^{t+\Delta t} = \mathbf{v}_i^t + \Delta t\, \mathbf{a}_i^t, \tag{5}$$

$$\mathbf{x}_i^{t+\Delta t} = \mathbf{x}_i^t + \Delta t\, \mathbf{v}_i^{t+\Delta t}. \tag{6}$$

L'ordre compte : la nouvelle vitesse sert à la position. À la fin du pas, appliquer la réflexion aux parois du domaine.

### 4.3 Référence exhaustive

Une invocation par particule, qui parcourt les $N-1$ autres :

```
#version 430 core
layout(local_size_x = 256) in;

layout(std430, binding = 0) readonly buffer Current {
    Particle current[];
};
layout(std430, binding = 1) writeonly buffer Next {
    Particle next[];
};

uniform uint N;
uniform float K;
uniform float eps2;
uniform float R;     // rayon d'interaction
uniform float dt;

void main() {
    // TODO : identifiant, garde, accumulation de (1),
    //        integration, reflexion aux parois, ecriture
}
```

### 4.4 Questions

1. Pourquoi cette version est-elle en $O(N^2)$ par pas de temps, et pourquoi le nombre d'invocations n'y change rien ?
2. Vérifier l'homogénéité de l'équation (4) : que se passe-t-il si l'on multiplie toutes les masses (ici 1) et $K$ par 2 ? Si l'on inverse tous les types $q_i$ ?
3. Pourquoi la version exhaustive et la version par grille doivent-elles appliquer exactement la même loi tronquée, y compris la réflexion aux parois ?

### 4.5 Tests de validation

— **Deux particules de même type**, immobiles, écartées de $d < R$ : elles doivent se rapprocher. L'écart doit décroître.

— **Deux particules de types différents** : elles doivent s'éloigner.

— **Inversion de tous les types** : la dynamique doit être identique (vérifier sur 100 pas).

— **Configuration symétrique** (4 particules aux sommets d'un carré) : le centre de masse doit rester immobile aux erreurs numériques près.

— Une particule isolée ($d > R$ de toutes les autres) doit garder une vitesse constante.

## 5 Exercice 3 : Particules, version spatiale

Squelette : `tp2/exercice3`. Deux shaders à compléter : `build_lists.comp` (construction des listes) et `query.comp` (forces, intégration, écriture de `next`). L'hôte remet déjà les têtes à $-1$, enchaîne les deux dispatchs et échange `current` et `next`.

Reprendre l'enchaînement de l'exercice 1. La construction des listes ne change pas, avec $h = R$. Seule la passe de forces change : à la place des trois règles des boids, accumuler l'équation (4). L'intégration, le ping-pong et la réflexion aux parois sont ceux de l'exercice 2. Comparer les positions à la référence exhaustive sur quelques dizaines de pas, puis le temps par pas quand $N$ augmente.

## Annexe : rappels de synchronisation

| **Situation** | **Outil** |
|---|---|
| publier des écritures shared au groupe | barrier () |
| ordonner deux dispatchs liés par SSBO | glMemoryBarrier( GL_SHADER_STORAGE_BARRIER_BIT) |
| rendu lisant un SSBO comme attributs de sommets | GL_VERTEX_ATTRIB_ARRAY_ BARRIER_BIT |
| insertion concurrente en tête de liste | atomicExchange |
| maximum global (occupation) | atomicMax |

Rappel : barrier () doit être rencontrée par *toutes* les invocations du groupe, dans le même ordre. Une sortie anticipée avant une barrière collective est un comportement indéfini.